

Three Roads: Choose Your Stack with AI

Ahmed Elgharably — FlyRank ML Internship — General AI Fluency, Week 4

The Four Constraints I Gave AI

Before asking for stack options, I wrote down my real constraints — not the ideal ones, the honest ones:

1. Free only

I have zero budget for hosting, domains, or subscriptions. Everything must be free-tier. If it asks for a credit card “just in case,” it’s out.

2. My honest skill level

- **Strong:** Python, data analysis, scikit-learn, Jupyter notebooks, Git, Markdown
- **Comfortable:** HTML/CSS basics, static site generators, GitHub Pages
- **Weak:** JavaScript frameworks (React, Vue), backend development (Node, Django), databases, DevOps, Docker
- **Learning:** Hugging Face, DuckDB, time-series ML

I can read and edit code, but I can’t build a full-stack app from scratch without AI holding my hand every step. I want to spend my time on ML work, not debugging a custom backend.

3. What my portfolio needs to do (from my content map)

- **Home page:** One-line claim, bio snapshot, proof preview
- **Projects page:** List of work, lead with FlyRank capstone
- **Case study page:** Deep dive into the capstone with code snippets, screenshots, results tables
- **About page:** Bio, headshot, beliefs, tools
- **Contact page:** Email, LinkedIn, GitHub

The portfolio is **text + images + links**. No user accounts, no forms, no database, no dynamic content that changes per visitor. The “dynamic” part is me updating it as I finish new assignments.

4. How my work must be displayed

- **Image galleries:** Screenshots of notebooks, EDA charts, model results tables
- **Embedded code:** Snippets from my notebooks (syntax highlighted)
- **Long-form reading:** Case studies are 800–1,500 words. They need clean typography and generous whitespace.
- **Code repo links:** Every project must link to the actual GitHub repo where the work lives
- **PDF downloads:** Some assignments are delivered as PDFs — visitors should be able to download them

Dynamic needs: None yet. If I build a live demo later (Week 7 capstone), I’ll cross that bridge then. For now, static is fine.

The Three Stack Options (AI-Generated, Then Curated)

I gave these constraints to Claude and asked for three options, simplest to most powerful. Here's what it produced, edited to my voice:

Option 1: GitHub Pages + Pure HTML/CSS

How you'd build: Write HTML files by hand or with AI assistance. One file per page (`index.html`, `projects.html`, `case-study.html`, etc.). CSS in a single `style.css` file.

Where you'd host: GitHub Pages — free, automatic deploy on every push to `main`.

Backend needed? No. Zero. None.

The trade-off:

- **Pros:** Completely free. No build step. No dependencies to break. I control every pixel. Fastest to ship. Directly tied to my repo.
 - **Cons:** Repetitive HTML (same header/footer on every page). No templating. Updating the navigation means editing every file. No automatic image optimization. Looks “hand-coded” which is either charming or amateur depending on the viewer.
 - **What breaks:** If I want to add a blog later, I'd need to copy-paste the template for every post. If I want a contact form, I can't — GitHub Pages is static only.
-

Option 2: GitHub Pages + Jekyll (or MkDocs)

How you'd build: Jekyll is a static site generator built into GitHub Pages. You write Markdown files, Jekyll converts them to HTML. Use a minimal theme or build your own layout matching the identity kit.

Where you'd host: GitHub Pages — Jekyll runs automatically on every push.

Backend needed? No. Jekyll builds the site at deploy time, not at request time.

The trade-off:

- **Pros:** Markdown instead of HTML — faster to write case studies. Templating means one header/footer file applies to all pages. Built-in syntax highlighting for code snippets. GitHub Pages supports Jekyll natively — no extra config.
 - **Cons:** Jekyll has a learning curve (YAML front matter, Liquid templating, `_config.yml`). Debugging build errors is annoying. Themes are either too generic or require heavy customization to match my identity kit. Ruby dependency if I want to preview locally.
 - **What breaks:** If Jekyll updates and breaks my theme, I'm debugging Ruby gems instead of writing ML assignments. If I want custom layouts beyond what Jekyll offers, I'm back to writing HTML inside Liquid templates — the worst of both worlds.
-

Option 3: Vercel + Next.js (or Astro)

How you'd build: Next.js (React) or Astro for a modern static site. Component-based architecture. Image optimization built-in. Can add dynamic features later (contact form API, search, etc.).

Where you'd host: Vercel — free tier for static sites, automatic deploy from GitHub.

Backend needed? Not yet. Both can export static HTML at build time. But the *platform* is designed for dynamic apps, so adding a backend later is trivial.

The trade-off:

- **Pros:** Modern, fast, professional. Component reuse means zero repetition. Automatic image optimization and lazy loading. Can add a contact form, search, or blog with tags later without rebuilding. Looks like a “real” developer portfolio.
 - **Cons:** JavaScript framework learning curve — I’m not strong here. Node.js ecosystem breaks constantly (dependency hell). Build errors are cryptic. Overkill for 5 pages of text and images. The temptation to add “just one more feature” is high, and every feature is time not spent on ML work.
 - **What breaks:** If I don’t touch the site for 3 months, `npm install` will probably fail. If Vercel changes their free tier policy, I might need to migrate. If I want to update a case study, I’m editing JSX components, not simple Markdown.
-

Pressure-Testing the Front-Runner: Option 1 (GitHub Pages + Pure HTML/CSS)

I leaned toward Option 1 because I’m already using it for the “Empty but Live” assignment. But I forced myself to answer the hard questions:

What breaks if I pick the simplest?

- **No templating:** I have to copy-paste the header, navigation, and footer into every HTML file. If I change the nav, I change 5 files. Mitigation: I only have 5 pages. The site is small enough that this is annoying, not crippling.
- **No Markdown:** I write case studies in Markdown, then convert to HTML. Mitigation: I can write in Markdown and use a simple converter (Pandoc, or even AI) to generate the HTML. Or I just write HTML directly — it’s not that much markup.
- **No automatic syntax highlighting:** Code snippets need `<pre><code>` blocks with a CSS library like Prism.js. Mitigation: One `<script>` tag in the head. Solved in 5 minutes.
- **No blog:** If I want to add posts later, each one is a new HTML file. Mitigation: I don’t need a blog. My portfolio is projects, not posts.

What do I maintain if I pick the most powerful (Option 3)?

- A `node_modules` folder with 500+ dependencies I don’t understand.
- A build pipeline that breaks when Node updates.
- JSX components that look simple but hide complexity (routing, state management, image pipelines).
- The constant temptation to add “just one more React library” for a fancy animation I don’t need.

Verdict: The maintenance cost of Option 3 is higher than the benefit for a 5-page static portfolio. I'm an ML engineer, not a frontend developer. My time is better spent on model validation, not `npm audit fix`.

Can I finish in two weeks?

- **Option 1:** Yes. I already have the blank page live. Filling 4 more pages is just writing HTML and copying content from my case studies. 2–3 evenings of work.
- **Option 2:** Maybe. Jekyll setup + theme customization + learning Liquid. Could take a week just to get the layout right. Then I still have to write the content.
- **Option 3:** No. Learning Next.js or Astro from scratch, fighting build errors, customizing components — that's a 3–4 week project by itself. My capstone would stall.

Does it show my work the way it needs to be shown?

- **Code snippets:** Yes — Prism.js handles this.
- **Image galleries:** Yes — simple `<img>` tags with CSS grid.
- **Long-form reading:** Yes — pure HTML/CSS gives me total control over typography. No theme fighting my identity kit.
- **PDF downloads:** Yes — direct `<a href="...">` links.
- **Repo links:** Yes — plain `<a>` tags.

Everything I need, Option 1 handles. Everything I don't need, Options 2 and 3 add.

My Decision

I chose Option 1: GitHub Pages + Pure HTML/CSS.

Why this one:

- **Free:** GitHub Pages is free forever with no credit card.
- **Maintainable:** I can open any HTML file and understand it. No build step, no dependencies, no framework to update. In 6 months, I can edit a page without relearning anything.
- **Fast to build:** The blank page is already live. Filling it is just writing content, not fighting tooling.
- **Shows my work well:** Total control over layout and typography. My identity kit (Space Grotesk, Inter, teal/stone palette) applies exactly as I designed it. No theme overriding my choices.
- **Honest about my skills:** I'm an ML engineer who can write HTML. I'm not a full-stack developer. A hand-coded portfolio signals "I build things that work" rather than "I copied a React template."

Why not Option 2 (Jekyll):

- Jekyll's "convenience" is actually friction for a site this small. I'd spend more time learning Liquid and debugging `_config.yml` than I'd save on copy-pasting headers.
- Markdown is nice, but I can write HTML almost as fast. The bottleneck is thinking, not typing tags.
- If Jekyll breaks, I'm debugging Ruby. I don't know Ruby. That's a dependency I can't afford.

Why not Option 3 (Next.js/Astro):

- Overkill. A bazooka for a mosquito. My portfolio has no dynamic content, no user accounts, no API calls, no search.
- The maintenance burden is real. Node.js projects rot fast. I'd spend capstone time fixing `package.json` instead of fixing model leakage.
- It sends the wrong signal. A React portfolio looks impressive but raises the question: "Why does an ML intern need a JavaScript framework for 5 pages of text?" The answer is: he doesn't.

The backend question:

Not yet. My portfolio is read-only. If I build a live demo for the capstone (Week 7), I'll use Hugging Face Spaces or Streamlit Cloud — tools designed for ML demos, not a custom backend. Adding a backend to a static portfolio is solving a problem I don't have.

The Honest Rationale

I asked AI to lay out my options. It did. Then I chose the one that matches my actual life, not my aspirational one. The simplest stack isn't a compromise — it's the right tool for the job. My portfolio's job is to show my ML work with clean typography and real evidence. HTML on GitHub Pages does that perfectly. Anything more is procrastination dressed up as engineering.

Submitted to the General AI Fluency track thread.